Forensic Acquisition and Analysis of Apple M4 Architecture: A Threat Hunter's Compendium

Section 1: The Modern Mac Forensic Landscape: Architecture and Principles

The introduction of Apple's M-series System on a Chip (SoC), culminating in the M4 architecture, represents a paradigm shift in computing security that necessitates a corresponding evolution in digital forensic methodologies. The deep integration of hardware and software security features has rendered many traditional forensic techniques obsolete, demanding a new approach grounded in a comprehensive understanding of the platform's design. This section deconstructs the M4 security model, establishes the foundational principles for modern macOS forensics, and outlines the prioritized strategy required to conduct a forensically sound investigation.

1.1 Deconstructing the M4 Security Model: A Forensic Arms Race

The security of an M4-based Mac is not an overlay but is woven into the silicon itself. This "security-by-design" philosophy begins with a hardware root of trust and extends through every layer of the boot process and operating system, creating a formidable challenge for forensic investigators.[1] Understanding this architecture is the first step toward overcoming the obstacles it presents.

Introduction to Apple Silicon Security

The M4 chip is more than a central processing unit; it is an integrated SoC that includes dedicated silicon for security functions.[2] This includes a hardware root of trust established by an immutable Boot ROM laid down during fabrication, dedicated AES cryptographic engines for high-speed data encryption, and the Secure Enclave Processor, which serves as the cornerstone of the device's security posture.[1] This tight coupling of hardware and software is designed to minimize the attack surface and protect the system's integrity from the moment it is powered on.

The Secure Enclave Processor (SEP): The Forensic "Black Box"

At the heart of the M4's security is the Secure Enclave Processor (SEP), a dedicated and isolated coprocessor that functions as a self-contained computer within the main SoC.[3] The SEP runs its own specialized microkernel, sepOS, and is completely segregated from the main Application Processor that runs macOS.[5] Its purpose is to handle all cryptographic operations and manage sensitive data, including FileVault encryption keys, biometric data for Touch ID, and other critical secrets.[1]

Crucially for forensic analysis, the keys managed by the SEP are *never* exposed to the Application Processor, its main memory (RAM), or the macOS kernel.[1] They are generated, stored, and used exclusively within the confines of the SEP. This fundamental design choice has profound implications for digital forensics. The integrated security architecture of the M4, with the SEP at its core, makes traditional "dead-box" forensics—where an investigator could simply power down a machine, remove the storage device, and create a bit-for-bit physical image—functionally impossible for the internal drive. The physical bond between the SEP and the SSD's encryption keys means the data is indecipherable if the drive is removed and connected to another system.[7] The only way to access decrypted data is through the M4 chip itself, which requires the system to be in an authenticated state. This reality forces a strategic shift away from passive, post-mortem acquisition and toward active, controlled interaction with the target device, making live analysis and forensically sound boot environments the primary methods of data collection.

Signed System Volume (SSV) and System Integrity Protection (SIP)

To further protect the operating system, modern macOS employs a Signed System Volume (SSV). During the boot process, the system cryptographically verifies the integrity of the entire system volume, ensuring that no unauthorized modifications have been made.[3] This

volume is mounted as read-only during normal operation. Layered on top of this is System Integrity Protection (SIP), which restricts even the root user from modifying critical system files and directories.[8] Together, SSV and SIP create a hardened environment that is highly resistant to tampering by malware and, by extension, complicates the use of forensic tools that require low-level system access.

Hardware-Accelerated Encryption and the T2/M-Series Legacy

The security model of the M4 builds upon the foundation established by the T2 security chip in earlier Intel-based Macs.[9] A core principle is that all data on the internal SSD is encrypted at the hardware level by default, using keys tied directly to the SEP's unique, unchangeable ID.[7] This hardware-accelerated encryption is performed by a dedicated AES engine, ensuring there is no performance penalty.[9] The consequence for forensics is stark: because the encryption is inextricably linked to the physical SEP on the logic board, physical acquisition of the SSD is forensically useless.

1.2 The Non-Negotiable Requirement: The Role of User Credentials

Given that the internal storage is always encrypted at the hardware level, accessing the user's data requires surmounting an additional layer of software encryption: FileVault 2.

FileVault 2 and SecureToken

FileVault 2 is the full-volume encryption feature in macOS that ties the hardware-based encryption keys to the user's login password. This is managed through the SecureToken framework, which ensures that only authenticated users can unlock the data volume.[11] When a user logs in, their password is used to unlock a key that, in turn, allows the SEP to release the volume encryption keys needed to decrypt the data on the fly.

The Password is the Key

The forensic implication of this architecture is absolute: without a valid user password, a FileVault recovery key, or another authorized SecureToken, meaningful data acquisition from an encrypted APFS container on a secure M4 Mac is impossible.[3] There are no known shortcuts or brute-force methods that can bypass the SEP's cryptographic protections. Therefore, a critical prerequisite for any investigation is the legal and procedural mechanism to obtain user credentials, whether through a search warrant compelling disclosure, corporate policy for employee devices, or other legal instruments.[3] Forensic readiness for a Mac environment now fundamentally includes planning for credential acquisition.

1.3 The Order of Volatility in the Apple Ecosystem: A Prioritized Collection Strategy

With the necessity of interacting with a live or booted system established, the collection of evidence must proceed according to a strict, universally accepted forensic principle: the Order of Volatility (OoV).[12] This principle dictates that evidence should be collected starting with the most ephemeral (short-lived) data and progressing to the most persistent (long-lived) data.[14] Adhering to the OoV is critical to prevent the loss of time-sensitive evidence, such as the contents of RAM or active network connections, which would be destroyed if a machine were prematurely powered down or rebooted.[12]

macOS-Specific OoV Hierarchy

The standard OoV must be adapted to the specific architecture and artifacts of an M4-based Mac. The hierarchy presented in Table 1 provides a systematic framework for data collection, ensuring that the most fragile evidence is captured first. This approach, grounded in standards like those from NIST and RFC 3227 [15], provides a defensible methodology for the entire data acquisition process.

Table 1: Order of Volatility for macOS M4 Systems

Volatility Rank	Data Type	Specific	Primary	Criticality/Rati

		macOS Artifacts	Collection Method/Tool	onale
1	CPU Registers & Cache	CPU state, active process information, kernel statistics.	sysctl -a, dmesg	Extremely volatile; data can be lost in milliseconds. Contains the immediate execution state of the system, which can be vital for advanced malware analysis. [14]
2	RAM & Running Processes	Full memory image, process tables, loaded kernel extensions, open files, clipboard contents.	OSXpmem, Cellebrite Digital Collector, ps aux, kextstat, lsof.	High volatility; lost on power-off. Contains in-memory decryption keys, malware components not on disk, passwords, and active C2 communicatio ns. [12]
3	Network State	Active network connections, ARP cache, routing tables, open ports.	netstat -anv, lsof -i, arp -a.	Very high volatility; connections can close or change in seconds. Essential for identifying attacker IP addresses and understanding

				lateral movement or data exfiltration. [12]
4	Temporary File Systems	Swap files (/private/var/vm/), temporary application/internet files.	Direct file copy, live imaging tools.	Moderate volatility; may persist across reboots but can be overwritten quickly. Can contain fragments of documents, passwords, or other user activity. [14]
5	Disk & Storage	Full disk image (APFS container), user files, system files, application data, logs.	Bootable forensic OS (e.g., RECON ITR), live logical acquisition tools.	Low volatility; data is persistent until intentionally deleted or overwritten. This is the primary source of long-term evidence. [13]
6	Remote Logging & Monitoring Data	Centralized logs (SIEM), cloud service logs (iCloud, Defender portal), network monitoring data.	Access to remote servers/portals.	Varies; depends on retention policies. Provides external context and corroboration for events observed on the endpoint. [14]

7	Physical Configuration & Network Topology	Hardware specifications, network diagrams, physical connections.	system_profiler, physical documentation, photography.	Very low volatility; changes infrequently. Provides context for the system's role and environment. [13]
8	Archival Media	Time Machine backups, external hard drives, cloud backups (e.g., iCloud).	Direct access to backup media/services, tmutil.	Extremely low volatility; designed for long-term persistence. Can provide historical snapshots of the system, crucial for recovering deleted data. [12]

Section 2: Phase 1: Live System & Volatile Data Acquisition

This phase details the immediate, hands-on procedures for capturing the most time-sensitive evidence from a running M4 Mac. Strict adherence to the Order of Volatility is paramount. The actions taken in the first few minutes of an investigation can determine whether critical evidence of an active compromise is preserved or irrevocably lost.

2.1 Memory Forensics on Apple Silicon: Tools, Techniques, and Limitations

The analysis of Random Access Memory (RAM) is one of the most critical steps in a modern

forensic investigation, particularly on heavily encrypted systems like M4 Macs. RAM can contain a wealth of volatile artifacts that may not exist on persistent storage, including active malware components, injected code, network packets, user passwords, and, most importantly, in-memory encryption keys that provide access to encrypted data.[12] For a system where the disk is encrypted by default, RAM analysis is not just valuable—it is often essential.[7]

Acquisition Tools

Acquiring a memory image from an Apple Silicon Mac presents significant technical challenges due to the hardened security architecture. While foundational open-source tools like OSXpmem (part of the Rekall suite) and older utilities like MacMemoryReader and volafox were effective on Intel-based Macs, their compatibility with the M1-M4 architecture is limited or non-existent.[15] The security controls in Apple Silicon often prevent direct, unfettered access to physical memory.

Consequently, investigators must typically rely on commercial forensic suites that have been specifically updated to support Apple Silicon. Tools such as Cellebrite Digital Collector (formerly Macquisition) and SUMURI RECON ITR have invested significant research and development into creating reliable methods for memory acquisition on the latest hardware.[20] These tools are often the only viable options for obtaining a forensically sound memory image from a live M4 system.

Analysis Frameworks

Once a memory image is acquired, it can be analyzed using powerful open-source frameworks like The Volatility Framework and Rekall.[17] These tools can parse the memory dump to reconstruct running processes, network connections, loaded kernel modules, and extract sensitive data like passwords and keys.[18]

However, a significant prerequisite for analysis is the creation of a memory "profile" that matches the exact version and build of the macOS kernel running on the target machine.[19] This process involves obtaining the corresponding Kernel Debug Kit (KDK) from Apple, extracting debugging symbols, and compiling them into a format that the analysis framework can understand.[19] This is a complex, error-prone task that can be a major bottleneck in an investigation. The lack of a correct profile will render the memory image largely unanalyzable.

Alternative Sources

In situations where live memory acquisition is not possible, investigators should examine the system for a sleepimage file, typically located at /var/vm/sleepimage.[15] When a Mac enters hibernation (safe sleep), the entire contents of RAM are written to this file so the system state can be restored. If present, this file can serve as a complete memory dump, offering the same forensic value as a live acquisition.

2.2 Capturing System State: The Power of Native Commands

Before a full memory or disk image is acquired, a series of native macOS commands must be executed to capture a snapshot of the system's immediate state. This process should be performed swiftly to minimize alterations to the live system. These commands can be bundled into a script and deployed from a trusted external USB drive to ensure consistency and speed.[23]

Initial Documentation

The first step upon approaching a live system, before any interaction, is to document its physical and logical state. The computer, its peripheral connections, and anything displayed on the screen should be photographed to create a record of the scene as it was found.[13]

Live Response Command Sequence

The following sequence of commands, executed from the Terminal, provides a baseline of critical system state information:

- **System Information:** Capture the current time, uptime, and detailed hardware/software configuration. This establishes a baseline for the investigation timeline and system context.
    - date [25]

    - uptime [26]
    - system_profiler SPSoftwareDataType SPHardwareDataType [26]
- **Logged-in Users & Sessions:** Identify who is currently logged in and review recent login history for anomalies.
    - who (Currently logged-in users) [27]
    - w (Current user activity and running commands) [27]
    - last (Historical login sessions) [27]
- **Running Processes:** Obtain a complete list of all running processes to identify suspicious or unauthorized activity.
    - ps aux [27]
- **Network Connections:** Document all active network connections and listening ports to identify potential command-and-control (C2) channels or data exfiltration paths.
    - netstat -anv [15]
    - lsof -i [28]
    - arp -a (ARP cache, showing recent local network interactions) [26]
- **Kernel State:** Dump kernel parameters and list loaded third-party kernel extensions (kexts), which can be a persistence mechanism for sophisticated malware.
    - sysctl -a [27]
    - kextstat | grep -v com.apple [27]

2.3 Leveraging Live Response Frameworks for Scalable Triage

In modern enterprise environments, physical access to an endpoint is often impractical or too slow. Live response frameworks provide security teams with immediate, remote access to devices, enabling real-time investigation and remediation. The combination of ubiquitous encryption and the richness of volatile data makes the ability to perform immediate, remote, and forensically sound live analysis a core competency for any modern DFIR team. The Order of Volatility dictates that the most crucial evidence of an active compromise, such as active network connections or in-memory malware, may exist for only seconds or minutes.[12] Physical response times are often measured in hours or days, by which point an attacker may have removed their tools or the system may have been rebooted, destroying all volatile evidence. Live response frameworks bridge this critical time gap, allowing for the instantaneous collection of this ephemeral data. Therefore, these tools are not merely for convenience; they are a direct countermeasure to the transient nature of modern threats and the security architecture of modern operating systems.

Microsoft Defender for Endpoint Live Response

Microsoft Defender for Endpoint includes a powerful live response feature that provides a remote shell connection to managed devices, including macOS.[29] This allows an analyst to perform in-depth investigative work from a central console. Key capabilities for macOS include:

- Running scripts from a library to automate data collection.
- Uploading forensic tools to the endpoint for specialized analysis.
- Collecting a forensic package containing logs and system information using the collect command.
- Downloading specific files of interest using the getfile command.[24]

Velociraptor

Velociraptor is an advanced open-source endpoint monitoring and DFIR platform that excels at rapid, scalable triage and data collection.[30] It uses the powerful Velociraptor Query Language (VQL) to define "artifacts"—reusable queries that can collect specific forensic data from thousands of endpoints simultaneously.

Velociraptor's artifact exchange provides a rich library of community-contributed artifacts for macOS, enabling investigators to quickly hunt for evidence of specific threats or collect standard forensic data sets.[31] This includes artifacts for parsing Safari history, collecting system logs, and even integrating with other forensic frameworks like Jamf's Aftermath to perform comprehensive data collection.[32] The ability to customize VQL allows threat hunters to adapt their collections to new and emerging threats in minutes.

Section 3: Phase 2: Persistent Storage & APFS Container Imaging

Following the collection of volatile data, the next phase of the investigation focuses on creating a forensically sound image of the Mac's internal persistent storage. This process is complicated by the Apple File System (APFS) and requires specialized tools and workflows that respect its unique architecture.

3.1 The Case for Native Tooling: asr, diskutil vs. Legacy Formats

The choice of imaging tool and format is one of the most critical decisions in a macOS investigation. Using inappropriate tools can lead to the omission or corruption of vital evidence, potentially invalidating the entire forensic analysis.

APFS Nuances

APFS is a modern, complex file system designed for solid-state storage. Its key features from a forensic perspective include the container model, where multiple logical volumes (e.g., System, Data, Preboot, Recovery) share space within a single physical container, and the use of point-in-time snapshots for backups and system updates.[11] Furthermore, APFS volumes are protected by cryptographic seals to ensure their integrity.[11]

Superiority of Native Commands

Apple-native command-line utilities such as asr (Apple Software Restore), diskutil, and hdiutil are purpose-built with a deep understanding of the APFS architecture. They can correctly interpret container structures, handle snapshot references, and preserve the cryptographic seals and metadata that are unique to macOS.[11] Leading commercial forensic tools, such as SUMURI RECON ITR and LLIMAGER, are effective precisely because they integrate and orchestrate these native commands within a forensically sound workflow, rather than attempting to reinvent them.[11]

Limitations of AFF4 and E01

Traditional forensic imaging formats like E01 (EnCase Evidence File) and AFF4 (Advanced Forensics File Format) are generally file-system-agnostic. While this is a strength in some contexts, it is a critical weakness when dealing with APFS. These formats lack the inherent awareness of APFS containers, snapshots, and Apple-specific metadata. An imaging tool

using these formats may fail to capture the complete and correct structure of the disk, leading to an incomplete or misinterpreted image.[3] Some tools attempt to overcome this by reverse-engineering the APFS format, but this approach is inherently risky and incomplete, as Apple does not publicly document the file system's internal structures.[11]

Preserving Apple Extended Metadata

A crucial advantage of using native tools is the preservation of Apple Extended Metadata. These are forensically rich attributes that are often stripped or ignored by non-native tools and formats. Key examples include:

- **Last Used Date:** A reliable timestamp indicating the last time a user directly interacted with a file or application, which can be more accurate for determining user intent than standard POSIX access times.[3]
- **Finder Tags:** Color-coded labels that users can apply to files, which may reveal organizational schemes or the perceived importance of data.[3]
- **Quarantine Flags:** Metadata attached to files downloaded from the internet, which includes the source URL and download timestamp. This is invaluable for tracing the initial vector of a malware infection.[3]

The loss of this metadata can lead to flawed investigative conclusions. For example, relying on a file's "Access Time" might incorrectly suggest a user opened a sensitive file, when in reality, the timestamp was updated by a background indexing service. The "Last Used Date" would provide the correct context.[3]

3.2 Forensically Sound Imaging Workflows

Creating a valid forensic image of an M4 Mac requires a carefully planned workflow. The chosen methodology will depend on the state of the system (powered on or off) and the available resources.

Hardware Write Blockers

A fundamental principle of digital forensics is to prevent any modification of the original

evidence. Hardware write blockers are devices that sit between the forensic workstation and the evidence media, enforcing read-only access at the hardware level. While essential, their application on modern Macs is nuanced. Since the internal SSD is not removable, a write blocker's primary role is in the acquisition of external drives connected to the target Mac. For imaging the internal drive via a bootable forensic OS, the operating system itself is responsible for mounting the internal drive in a read-only state to ensure forensic soundness. A wide range of NIST-tested write blockers are available from vendors like Logicube, CRU WiebeTech, and Digital Intelligence.[36]

Imaging Methodologies

There are several methods for imaging an M4 Mac, each with distinct advantages and forensic considerations. Table 2 provides a comparative overview to guide the investigator's choice.

Table 2: Comparison of Apple Silicon Imaging Methodologies

Methodology	Description	Pros	Cons	Credential Requirement	Forensic Soundness	Recommended Use Case
Bootable Forensic OS	Booting the target Mac from a trusted, external macOS-based forensic environment. [3]	- Most forensically sound method. - Captures the entire APFS container, including all volumes and snapshots.	- Requires rebooting the target system, resulting in the loss of volatile data (RAM, etc.). - Requires a pre-confi	Admin password or FileVault recovery key required to unlock the data volume. [40]	**High**	Full, in-depth evidence collection for criminal cases, corporate investigations, and any scenario requiring the most complete

		- Preserves all Apple Extended Metadata. 11	gured forensic boot drive.			data set.
Live Logical Acquisition	Acquiring data from the target Mac while it is running its native operating system. [3]	- Minimal disruption to the system. - Does not require a reboot, preserving system uptime. - Can be performed remotely. 7	- Not a bit-for-bit image; acquires files at the logical level. - Data collection is limited by the permissions of the logged-in user. - The act of running the acquisition tool alters the state of the live system. 7	Admin password required for the broadest access to system and other user files.	**Medium**	Rapid triage in corporate environments where business continuity is a priority and a full disk image is not immediately required.
Share Disk Mode	Booting the target Mac into macOS Recovery and	- Does not require a specialized forensic boot	- Extremely slow and unreliable for large data	Admin password required to mount and decrypt the	**Low**	Not recommended for forensic imaging. May be a last

	sharing its internal disk over the network to another Mac. [3]	drive.	transfers. - Connections are prone to dropping, potentially corrupting the image. - Not designed for forensic-grade acquisition. 3	shared volume.		resort for very small, targeted data transfers if other methods fail.

- ---

  Method 1: Bootable Forensic Environment (Preferred): This is the gold standard for acquiring a complete and forensically sound image. The process involves starting the Mac while holding the Option key to enter the Startup Manager, selecting the external forensic drive (e.g., one running SUMURI RECON ITR or ADF Digital Evidence Investigator), and booting into the controlled environment.[40] Once booted, the investigator provides the user's credentials to unlock the internal FileVault 2 volume. The forensic software can then create a complete image of the APFS container, preserving its complex structure and metadata, and save it to a sterile external evidence drive.[3]
- **Method 2: Live Logical Acquisition:** This method is suitable for situations where shutting down the system is undesirable. Forensic tools like LLIMAGER or Exterro FTK Enterprise are run directly on the live system to copy files and artifacts.[7] While convenient, this method has significant forensic drawbacks. It is not a true physical image, and the acquisition process itself will alter the system by creating processes, writing to logs, and modifying file access times. Its use should be carefully documented and justified.
- **Method 3: Share Disk Mode (Not Recommended):** This feature, accessible through macOS Recovery, allows the internal disk to be shared as a network volume. While technically a method of data transfer, it is notoriously slow and unstable, making it unsuitable for the large-scale data transfer required for a full forensic image.[3] Its use should be avoided in favor of more reliable methods.
- **Method 4: Device Firmware Update (DFU) Mode:** DFU mode is a low-level state used primarily for firmware recovery and restoration.[42] It allows a Mac to be connected to

another Mac to have its firmware revived or reinstalled.[43] While it establishes a direct, low-level connection between two machines, it is not a standard or practical method for forensic disk imaging and is more relevant to the advanced firmware analysis discussed in the next section.[25]

Section 4: Phase 3: Firmware and Low-Level Component Interrogation

As operating system-level security controls become increasingly robust, sophisticated adversaries are shifting their focus to the firmware layer. A compromise at this level, often in the form of a bootkit, can achieve unparalleled stealth and persistence by executing before the operating system and its security tools are loaded.[44] Therefore, a comprehensive forensic examination of a system targeted by an advanced threat actor must include an interrogation of the firmware and boot process. This represents a new frontier for many DFIR investigators, requiring specialized tools and knowledge to uncover threats that are invisible to traditional disk and memory analysis.

4.1 Analyzing the Boot Chain: From Boot ROM to iBoot and Kernel

The macOS boot process is a meticulously designed "chain of trust," where each stage cryptographically verifies the integrity of the next before passing execution control. This process is designed to prevent unauthorized code from running at any point during startup.[45]

- **Chain of Trust:** The process begins with the immutable **Boot ROM**, which is the hardware root of trust burned into the Apple Silicon SoC at the time of manufacture.[1] The Boot ROM loads and verifies the **Low-Level Bootloader (LLB)**. The LLB, in turn, verifies and loads **iBoot**, the stage 2 bootloader for all modern Apple devices.[45] Finally, iBoot is responsible for verifying the digital signature of the macOS kernel and loading it into memory to start the operating system.[47] A failure at any stage of this verification process will halt the boot-up, preventing the system from starting with compromised code.
- **Forensic Significance:** A bootkit is malware designed to infect this boot chain, typically by modifying iBoot or the kernel. By doing so, it can subvert the entire security model of macOS, bypassing SIP, SSV, and any endpoint security software.[44] Detecting such threats requires analyzing the components of the boot chain themselves.

- **BFU/AFU States:** The concepts of "Before First Unlock" (BFU) and "After First Unlock" (AFU), originating from iOS forensics, are relevant here.[51] In a BFU state, the device is powered on, but a user has not yet entered their passcode. In this state, the vast majority of user data remains encrypted and inaccessible. Forensic activities may be limited to observing the device's state and, if possible, examining low-level components like the iBoot firmware, as this may be one of the few accessible artifacts.[51]

4.2 Methods for Firmware Extraction and Analysis

Analyzing firmware requires first extracting it from the system or from update packages to create a baseline for comparison.

- **Extraction from macOS Installers:** A "clean" or known-good version of the firmware can be extracted from official macOS update packages (.pkg files). Using command-line utilities like pkgutil to expand the package and unzip to extract the firmware payload (typically a .scap file), an analyst can obtain a pristine copy of the firmware for a specific Mac model and OS version.[52] This baseline is essential for comparison against the firmware extracted from a potentially compromised machine.
- **UEFI Memory Forensics:** A cutting-edge technique involves capturing and analyzing the volatile memory of the UEFI environment *during* the pre-boot phase.[44] This can reveal malicious modifications, such as function pointer hooking or inline code hooks, that are active only during boot and would be invisible to OS-level memory analysis.[44] This is a highly specialized field but represents the future of detecting advanced firmware threats.
- **Firmware Analysis Tools:** Once a firmware image is obtained, several tools can be used to parse and analyze its contents:
    - **Open Source:** Tools like **UEFITool** and its command-line counterpart **UEFIExtract** are invaluable for parsing UEFI-compliant firmware images, allowing an analyst to explore their structure and extract individual components like drivers and applications.[53]
      FwHunt is a tool specifically designed to scan firmware images against a set of rules to detect known vulnerabilities and malicious implants.[55]
    - **Cloud-Based Services:** Platforms like Microsoft's firmware analysis feature in Defender for IoT can automatically analyze uploaded firmware images, checking for known CVEs, weak binary hardening configurations, and embedded cryptographic keys or passwords.[56]
    - **Live Scanning:** Modern endpoint security solutions are beginning to incorporate firmware scanning. **Microsoft Defender for Endpoint** includes a UEFI scanner that can perform dynamic analysis on the firmware file system at runtime.[57] Similarly, **Velociraptor** can be used to collect EFI signature information from live systems,

which can be used for threat hunting and identifying anomalies across a fleet of endpoints.[58]

4.3 The Secure Enclave: Understanding the Forensic "Black Box"

While firmware analysis is becoming more accessible, it is crucial to reiterate the forensic limitations of the Secure Enclave. The SEP is designed from the ground up to be an impenetrable, isolated subsystem.[4] There are currently no known public software-based methods to directly access the SEP's internal memory, inspect its running processes, or extract the cryptographic keys it manages.[6]

Forensic analysis is therefore limited to observing the *results* of the SEP's operations—for example, a volume being successfully decrypted after a password is provided—rather than the operations themselves. The SEP's internal workings remain a "black box" to the investigator. This is a fundamental architectural constraint that must be acknowledged in any forensic report. Analysis must focus on the artifacts available on the Application Processor and the main operating system, which are made accessible *by* the SEP's functions.[1]

Section 5: Phase 4: Granular Artifact Extraction and Analysis

This section provides a comprehensive catalog of the most critical forensic artifacts within macOS. Following the acquisition of a system image, the focus shifts to the meticulous extraction and analysis of the data structures, databases, and log files that record system and user activity. Modern macOS forensics has moved beyond parsing individual flat files; it now centers on querying structured, inter-related data sources. A single user action, such as downloading and executing a malicious application, will create a trail of evidence across multiple, distinct artifacts. A successful investigation hinges on the ability to connect these disparate pieces of evidence to construct a coherent timeline of events.

Subsection 5.1: System Activity and Timestamps

These artifacts provide a high-level overview of system-wide events and are foundational for building an investigative timeline.

Apple Unified Log (AUL)

The Apple Unified Log is the centralized logging system for macOS, replacing the collection of plaintext logs from older OS versions. It is an invaluable, albeit dense, source of information.

- **Location & Structure:** The AUL stores log data in a compressed, binary format in a series of directories, primarily /var/db/diagnostics/ and /var/db/uuidtext/.[59] Direct manual parsing is not feasible.
- **Collection:** The preferred method for collection is using the command sudo log collect, which gathers all available log data into a single .logarchive file. This file can then be transferred to a forensic workstation for analysis. For live analysis, log stream can be used to view events in real time.[62]
- **Analysis:** Analysis is performed using the log show command, which requires the use of predicate-based filtering to narrow down the millions of log entries to relevant events. Effective use of predicates is the key to unlocking the value of the AUL.[62]

Table 3: Essential Unified Log Predicates for Threat Hunting

Investigative Category	Example Predicate	Evidentiary Value
User Logins	processImagePath contains "loginwindow" && eventMessage contains "com.apple.sessionDidLogin"	Tracks successful graphical user interface (GUI) logins, providing a timeline of user access. [65]
Privilege Escalation	process == "sudo" && eventMessage CONTAINS "COMMAND="	Records commands executed with elevated privileges via sudo, often a key step for attackers. [66]
Software Installation	subsystem == "com.apple.mac.install"	Logs events related to the installation of new software packages and updates. [64]

External Device Connections	subsystem == "com.apple.filesystems.hfs" OR subsystem CONTAINS "USB"	Can reveal the connection and mounting of external USB storage devices. Searching for USBMSC is also effective. [67]
Network Activity (SSH)	process == "sshd"	Captures successful and failed SSH login attempts, indicating remote access activity. [66]
Persistence (Kext)	process == "kextd" && sender == "IOKit"	Tracks the loading of kernel extensions, a powerful persistence mechanism for advanced malware. [66]
Gatekeeper/XProtect	process == "syspolicyd" OR subsystem == "com.apple.xprotect"	Shows events related to Gatekeeper (code execution policy) and XProtect (malware signature scanning). [64]
System Shutdowns	eventMessage contains "Previous shutdown cause"	Helps identify the reason for system reboots, which can be useful for diagnosing crashes caused by malware. [63]

FSEvents (.fseventsd)

FSEvents provide a detailed log of file system activity.

- **Purpose & Location:** The fseventsd daemon monitors and logs file system modifications (e.g., creation, deletion, renaming, modification) for each volume. The logs are stored in a hidden .fseventsd directory at the root of the volume.[69]
- **Structure:** The logs are stored as a series of gzipped files. Each record within a decompressed log contains the full path of the modified file, an event ID, and a set of event flags that describe the action performed. The file format is identified by a magic header, which has evolved from 1SLD and 2SLD in older versions to 3SLD in modern macOS.[69] Interpreting the event flags is key to understanding the activity; for example,

specific flags indicate whether a file was created, renamed, or moved to the Trash.[70]
- **Parsing:** Due to the complex format, manual analysis is impractical. Specialized parsers, such as the open-source FSEventsParser scripts or the mac_apt tool, are required to convert the logs into a human-readable format.[69]

Spotlight Database (.mdsstores)

Spotlight is the system-wide search service in macOS, and its index can be a source of forensic artifacts.

- **Purpose:** Spotlight's daemons (mds and mds_stores) constantly index file contents and metadata to enable fast searches.[72]
- **Forensic Value:** The most significant value of the Spotlight database is its potential to retain metadata about files that have been deleted from the file system. An investigator may find the name, path, and other attributes of a deleted file within the Spotlight index, providing crucial leads even after the file itself is gone.[73]
- **Location & Analysis:** The main system index is located at /private/var/db/Spotlight-V100/, with per-user indexes also present in user home directories. The mdls (metadata list) command-line utility can be used to query the Spotlight metadata for a specific file.[28] While reindexing Spotlight can resolve performance issues, this action should be avoided during an investigation as it will destroy potential evidence.[72]

Time Machine & APFS Snapshots

Time Machine and the underlying APFS snapshot feature provide a powerful mechanism for data recovery and historical analysis.

- **Functionality:** APFS can create lightweight, point-in-time snapshots of a volume. Time Machine leverages this to create local snapshots on the internal drive, in addition to its regular backups to an external drive or network location.[76]
- **Forensic Value:** These snapshots are forensically invaluable. They can be used to recover previous versions of files that have been modified or encrypted by ransomware, or to recover files that were deleted by an attacker in an attempt to cover their tracks.[77]
- **Analysis:** The tmutil command-line utility is the primary tool for interacting with Time Machine and snapshots. It can be used to list backup destinations (destinationinfo), enumerate all available backups (listbackups), and, most importantly, list local snapshots

stored on the system drive (listlocalsnapshots /).[28] Mounting and analyzing these snapshots can be complex; commercial tools like Blacklight can assist in visualizing the backup history, but extreme care must be taken not to modify the original backup evidence.[79]

Subsection 5.2: Execution and Persistence

This group of artifacts reveals how malicious code was executed and how it configured itself to survive a system reboot.

Launch Agents & Daemons

This has long been the primary persistence mechanism on macOS.

- **Mechanism:** launchd, the system's service manager, automatically launches programs based on configuration files known as Property Lists (.plist files).[27] Launch Daemons run at the system level, while Launch Agents run on a per-user basis.
- **Locations:** The key directories to examine are /Library/LaunchAgents, /Library/LaunchDaemons, and the user-specific ~/Library/LaunchAgents directory.[80]
- **Analysis:** The launchctl list command can show currently loaded jobs.[27] However, a more thorough approach is to use a tool like Objective-See's KnockKnock, which is specifically designed to enumerate all known persistence locations on macOS and flag suspicious entries.[82]

Background Task Management (BTM) Database (macOS 13+)

Starting with macOS 13 Ventura, Apple consolidated the management of persistence items into a new subsystem.

- **The New Standard:** The Background Task Management (BTM) system now centrally manages Launch Agents, Launch Daemons, and Login Items.[80]
- **Location & Analysis:** The subsystem maintains a database at /private/var/db/com.apple.backgroundtaskmanagement/. While the database itself is a binary plist, the built-in sfltool utility can be used to dump its contents in a

human-readable format, providing a single, authoritative source for most persistence mechanisms on a modern system.[80]

Quarantine Events

This artifact is critical for tracing the origin of externally sourced files.

- **Purpose:** When a file is downloaded from the internet by an application like a web browser or email client, macOS applies a "quarantine" extended attribute to it.[26]
- **Database:** Details of these events are logged in a SQLite database named QuarantineEventsV2, located at ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2.[83]
- **Forensic Value:** This database is forensic gold. Each record contains the timestamp of the download, the source URL, and the name and bundle ID of the agent that performed the download (e.g., Safari, Chrome). This information is often the key to identifying the initial infection vector for malware.[83] The database can be parsed effectively using tools like Velociraptor.[83]

Shell History

The command-line history provides a direct record of an operator's actions.

- **Files:** By default, macOS uses the Z shell (Zsh), which stores its history in the ~/.zsh_history file. If the user has switched to Bash, the history will be in ~/.bash_history.[27]
- **Forensic Value:** This file contains a log of the commands executed in the terminal by the user or an attacker. It can reveal the tools used, files accessed, and the sequence of actions taken during a compromise.[27] The shell's configuration file (e.g., ~/.zshrc) can be modified to increase the number of history entries saved and to add timestamps to each command, enhancing its forensic value.[85]

Subsection 5.3: User Activity and Configuration

These artifacts provide deep insight into how a user interacted with the system and its

applications.

knowledgeC.db and the Biome Database

For years, the knowledgeC.db database was a primary source for detailed user activity.

- **Purpose:** This SQLite database tracked a vast range of user interactions, including application usage (focus, duration), device state changes (locking, unlocking, charging), browsing history, and more.[86]
- **Location:** The database is located at ~/Library/Coreduet/Knowledge/ on macOS.[88]
- **Critical Nuance:** It is essential for investigators to know that starting with macOS 13 Ventura (and iOS 16), Apple has migrated the vast majority of this tracking functionality to a new, more complex data store known as the **Biome database**. On an M4 Mac running a current version of macOS, knowledgeC.db will be of limited value. The focus of the investigation for this type of pattern-of-life analysis must shift to parsing the Biome database.[86]

Transparency, Consent, and Control (TCC.db)

This database is the gatekeeper for user privacy permissions.

- **Purpose:** TCC.db is a SQLite database that records which applications have been granted user consent to access protected resources like the camera, microphone, location services, contacts, and Full Disk Access.[28]
- **Location:** There are separate databases for system-wide (/Library/Application Support/com.apple.TCC/) and user-specific (~/Library/Application Support/com.apple.TCC/) permissions.[28]
- **Analysis:** A query of the access table is crucial. It reveals the client (the application), the service (the permission requested), and a last_modified timestamp. This can show if a user was tricked into granting a malicious application sensitive permissions.[60]

Keychain

The macOS Keychain is the secure, encrypted repository for user credentials.

- **Purpose:** The Keychain stores a wide variety of secrets, including website passwords, Wi-Fi passwords, application-specific tokens, private keys, and secure notes.[89]
- **Location:** Keychain database files are stored in the ~/Library/Keychains/ directory.[89]
- **Forensic Value:** The Keychain is a primary target for information-stealing malware.[28] The database is implemented as a SQLite file, but its contents are heavily encrypted using AES-256-GCM, with the encryption keys ultimately protected by the Secure Enclave.[89] Accessing the decrypted contents requires either the user's login password or advanced password cracking techniques. Forensic analysis often focuses on extracting the Keychain file and attempting an offline attack on the user's password to unlock it.[91]

Property List (Plist) Files

Plists are the macOS equivalent of the Windows Registry, storing configuration settings for the OS and applications.

- **Purpose & Format:** These files are scattered throughout the file system and store settings as key-value pairs. They can be in either a human-readable XML format or a binary format that requires a parser.[71]
- **Key Plists for Forensics:** While there are thousands of plists, certain ones are of high forensic value. Examples include:
  - com.apple.wifi.known-networks.plist: Contains a history of all Wi-Fi networks the Mac has connected to, including SSID and connection timestamps.[92]
  - com.apple.loginwindow.plist: Records the last logged-in user and other session information.[81]
  - com.apple.sidebar.plist: May contain the names of recently mounted external volumes.[93]

Application Support & Containers

These directories are a treasure trove of application-specific data.

- **Location:** The primary locations are ~/Library/Application Support/ and, for sandboxed apps, ~/Library/Containers/.
- **Forensic Value:** These directories contain the databases, caches, logs, and configuration files for nearly every application installed on the system. An investigation into user activity within a specific application (e.g., a web browser, messaging client, or productivity tool) will almost always involve a deep dive into its corresponding folder

within these locations.[94]

Subsection 5.4: Connectivity and External Devices

Understanding what external devices have been connected to a Mac can be critical for investigating data exfiltration or the introduction of malware.

USB/Thunderbolt Connection History

- **Primary Source:** A common misconception among investigators familiar with Windows is that macOS maintains a persistent, centralized registry of all previously connected USB devices. This is not the case.[96] There is no direct equivalent to the Windows USBSTOR registry key.
- **Unified Log Analysis:** The most reliable source for historical USB and Thunderbolt device connection information is the Unified Log. An investigator must query the AUL using specific predicates to search for messages related to the USB Mass Storage Class (USBMSC) driver or other relevant subsystems to piece together a connection history.[67]
- **Live System Information:** For devices that are currently connected, the system_profiler command can provide detailed information. The SPUSBDataType and SPThunderboltDataType arguments will list all connected devices, their vendor and product IDs, and serial numbers.[68]
- **Plist Artifacts:** Some secondary artifacts may provide clues. Plist files such as com.apple.sidebar.plist and com.apple.finder.plist may cache the volume names of previously mounted external drives, but this information is often incomplete and lacks the detailed hardware identifiers found in the Unified Log.[93]

Section 6: Phase 5: Synthesis, Timeline Construction, and Reporting

The final phase of the investigation involves synthesizing the vast amounts of data collected from disparate artifacts into a coherent narrative, analyzing any recovered malicious code, and presenting the findings in a clear, defensible, and forensically sound report.

6.1 Correlating Evidence: Building a Master Timeline of Events

The ultimate goal of forensic analysis is to reconstruct the sequence of events related to an incident. This is achieved by creating a master timeline that correlates timestamps from every piece of evidence collected.

- **The Goal of Timeline Analysis:** A timeline provides a chronological view of all system and user activity, allowing an investigator to see what happened before, during, and after a security event. It can reveal cause-and-effect relationships, such as a file download followed by a process execution, followed by a network connection to a malicious IP address.
- **Tools for Timeline Creation:** Manually correlating timestamps from hundreds of different log files and databases is an impossible task. Automated timeline creation tools are essential. Open-source frameworks like **Plaso (Log2timeline)** are designed to parse a wide variety of forensic artifacts from a disk image and extract all available timestamps into a single, massive timeline file.[102] Commercial forensic suites like **Belkasoft X** have powerful, integrated timeline analysis features that can ingest data from multiple sources (computers, mobile devices, cloud) and visualize the events on an interactive timeline.[104] Tools like Eric Zimmerman's **Timeline Explorer** are then used to load, filter, sort, and analyze the generated timeline files, allowing an investigator to pivot through the data and focus on periods of interest.[105]
- **The Process:** The workflow involves pointing the timeline creation tool at the acquired disk image, memory dump, and any exported log files. The tool parses all known artifacts and outputs a unified timeline, typically in CSV or a similar format. The analyst then uses a timeline viewer to navigate this data, searching for key events, filtering by time ranges, and building a narrative of the incident.

6.2 Static and Dynamic Analysis of Recovered Malicious Payloads

If the investigation uncovers suspicious executable files, a malware analysis phase is required to understand their purpose and capabilities.

- **Malware Analysis Overview:** The primary goals of malware analysis are to determine the functionality of the malicious code, extract Indicators of Compromise (IOCs) such as file hashes, C2 domains, and registry keys (or plist modifications), and develop signatures for future detection.[106]

- **Static Analysis:** This involves examining the malware binary without executing it. On macOS, this includes extracting embedded strings, checking the code-signing certificate and entitlements (using a tool like Objective-See's What's Your Sign), and performing disassembly to analyze the code's logic.[82]
- **Dynamic Analysis:** This involves running the malware in a safe, isolated environment (a sandbox) to observe its behavior. This is a powerful technique for uncovering the malware's true purpose. Tools from the Objective-See suite are invaluable here: FileMonitor can track all file system modifications, ProcessMonitor can observe process creation and inter-process communication, and LuLu can monitor and block network connections.[82] For more automated analysis, open-source sandboxes like Cuckoo can be configured to analyze macOS malware.[108]
- **macOS Malware Trends:** It is crucial to contextualize findings with current threat intelligence. Recent trends in macOS malware include a significant rise in information stealers that are socially engineered to bypass Gatekeeper. These stealers often target sensitive user data, including Keychain files for passwords, browser credentials and cookies, and cryptocurrency wallet files.[109]

6.3 Maintaining the Chain of Custody and Presenting Forensic Findings

The final output of an investigation is only as strong as the process used to generate it. Meticulous documentation and adherence to forensic principles are non-negotiable.

- **Chain of Custody (CoC):** The Chain of Custody is the chronological documentation that tracks the movement and handling of evidence throughout its lifecycle—from collection and safeguarding to analysis and presentation.[110] According to NIST guidelines, every action performed on the evidence must be documented, including who handled it, the date and time of the transfer, and the purpose for the transfer.[110] This includes everything from connecting a hardware write blocker to running a live response script. The integrity of all collected digital evidence (disk images, memory dumps, exported logs) must be proven by calculating and recording cryptographic hashes (e.g., SHA-256) at the time of acquisition and verifying them at each step of the analysis.[28] Standardized Chain of Custody forms should be used for all evidence handling.[111]
- **Reporting:** The final forensic report is the culmination of the entire investigative effort. It must present the findings in a clear, factual, and objective manner. The report should detail the scope of the investigation, the methodologies and tools used, a summary of the findings, and a detailed timeline of events that reconstructs the incident. This timeline, supported by the evidence from the various artifacts analyzed, provides the definitive narrative that allows all involved parties to understand the root cause of the

incident and make informed decisions about remediation and future prevention.

Works cited

1. Apple Platform Security, accessed on September 5, 2025, https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf
2. Apple introduces M4 Pro and M4 Max, accessed on September 5, 2025, https://www.apple.com/newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/
3. Imaging Apple Silicon Macs: A Modern Forensic Guide : SUMURI, accessed on September 5, 2025, https://sumuri.com/imaging-apple-silicon-macs-a-modern-forensic-guide/
4. Secure Enclave - Apple Support, accessed on September 5, 2025, https://support.apple.com/guide/security/secure-enclave-sec59b0b31ff/web
5. Do M4 chips have a new security processor? - The Eclectic Light Company, accessed on September 5, 2025, https://eclecticlight.co/2025/02/13/do-m4-chips-have-a-new-security-processor/
6. How the Secure Enclave protects your Mac - The Eclectic Light Company, accessed on September 5, 2025, https://eclecticlight.co/2023/08/21/how-the-secure-enclave-protects-your-mac/
7. Forensic Collections From a Mac: Challenges & Solutions - Cyber Defense Magazine, accessed on September 5, 2025, https://www.cyberdefensemagazine.com/forensic-collections-from-a-mac-challenges-solutions/
8. macOS Forensics: The Basics - TryHackMe Walkthrough | by RosanaFSS | Medium, accessed on September 5, 2025, https://medium.com/@RosanaFS/macos-forensics-the-basics-tryhackme-walkthrough-35b6a567db9b
9. Overview of Mac system security and its impact on digital forensics process - ResearchGate, accessed on September 5, 2025, https://www.researchgate.net/publication/346751841_Overview_of_Mac_system_security_and_its_impact_on_digital_forensics_process
10. From FileVault to T2: How to Deal with Native Apple Encryption - Passware Support, accessed on September 5, 2025, https://support.passware.com/hc/en-us/articles/21317262862231-From-FileVault-to-T2-How-to-Deal-with-Native-Apple-Encryption
11. macOS Native Imaging vs AFF4 for APFS Forensics : SUMURI, accessed on September 5, 2025, https://sumuri.com/why-macos-native-imaging-commands-are-preferred-over-aff4-for-apfs-forensics/
12. The Order Of Volatility In A Data Breach - Responsum, accessed on September 5, 2025, https://responsum.eu/articles/what-is-the-order-of-volatility-in-a-data-breach/
13. Digital Forensics: Acquiring Evidence - MCSI Library, accessed on September 5, 2025,

https://library.mosse-institute.com/articles/2023/09/digital-forensics-acquiring-evidence.html
14. What is Order of Volatility? - Clocked-Out, accessed on September 5, 2025, https://clocked-out.com/what-is-order-of-volatility/
15. Mac OS X Memory Analysis: An Overview - scip AG, accessed on September 5, 2025, https://www.scip.ch/en/?labs.20120719
16. How would you answer this question on Order of Volatility? : r/CompTIA - Reddit, accessed on September 5, 2025, https://www.reddit.com/r/CompTIA/comments/8hx9jm/how_would_you_answer_this_question_on_order_of/
17. Memory Forensics and the Macintosh OS X Operating System - Faculty - Naval Postgraduate School, accessed on September 5, 2025, https://faculty.nps.edu/ncrowe/oldstudents/icdf2c17_leopard.htm
18. Memory forensics | Network Security and Forensics Class Notes - Fiveable, accessed on September 5, 2025, https://library.fiveable.me/network-security-and-forensics/unit-5/memory-forensics/study-guide/jONCvd0TLEOPcT74
19. OSX (Mac) Memory Acquisition and Analysis Using OSXpmem and Volatility | Ponder The Bits, accessed on September 5, 2025, https://ponderthebits.com/2017/02/osx-mac-memory-acquisition-and-analysis-using-osxpmem-and-volatility/
20. M2 Mac Forensic Imaging Tools any better than simple built-in Disk Utility Read-only Image? - Reddit, accessed on September 5, 2025, https://www.reddit.com/r/computerforensics/comments/17lei3f/m2_mac_forensic_imaging_tools_any_better_than/
21. Industry First: Explore the Latest Features in Forensic Imaging Solutions, accessed on September 5, 2025, https://enterprise.cellebrite.com/industry-first-explore-the-latest-features-in-forensic-imaging-solutions/
22. Improving Memory Forensics Capabilities on Apple M1 Computers - LSU Scholarly Repository, accessed on September 5, 2025, https://repository.lsu.edu/context/gradschool_theses/article/6610/viewcontent/REVISEDRaphaela_Thesis.pdf
23. Tools - BriMor Labs, accessed on September 5, 2025, https://www.brimorlabs.com/tools/
24. Collect support logs in Microsoft Defender for Endpoint using live response, accessed on September 5, 2025, https://learn.microsoft.com/en-us/defender-endpoint/troubleshoot-collect-support-log
25. Forensic Acquisition of Mac Computers, accessed on September 5, 2025, https://www.computerpi.com/forensic-acquisition-of-mac-computers/
26. Apples to Apples: Why macOS Forensics Can Be Easier Than Windows - Medium, accessed on September 5, 2025, https://medium.com/@mathias.fuchs/apples-to-apples-why-macos-forensics-can-be-easier-than-windows-19c9f234c1a1

27. macOS Forensics Artifacts and Commands - CyberDom, accessed on September 5, 2025, https://cyberdom.blog/macos-forensics-artifacts-and-commands/
28. MAC Device Forensics, accessed on September 5, 2025, https://rokibulroni.com/blog/mac-forensics/
29. Investigate entities on devices using live response in Microsoft Defender for Endpoint, accessed on September 5, 2025, https://learn.microsoft.com/en-us/defender-endpoint/live-response
30. Velociraptor: Digital Forensics & Incident Response (DFIR) - Rapid7, accessed on September 5, 2025, https://www.rapid7.com/products/velociraptor/
31. Artifact Exchange :: Velociraptor - Digging deeper!, accessed on September 5, 2025, https://docs.velociraptor.app/exchange/
32. MacOS.Collection.Aftermath :: Velociraptor - Digging deeper!, accessed on September 5, 2025, https://docs.velociraptor.app/exchange/artifacts/pages/macos.collection.aftermath/
33. Custom artifacts for Velociraptor - GitHub, accessed on September 5, 2025, https://github.com/Seeps/Velociraptor-Artifacts
34. What is APFS (Apple File System)? | Our Definition - MSAB, accessed on September 5, 2025, https://www.msab.com/glossary/apfs-apple-file-system/
35. LLIMAGER - macOS forensic imager, accessed on September 5, 2025, https://www.llimager.com/
36. WriteProtect™-DESKTOP - Logicube, accessed on September 5, 2025, https://www.logicube.com/shop/writeprotect-desktop/
37. Forensic Write-Blocking Drive Docks | CDSG - CRU Data Security Group, accessed on September 5, 2025, https://cdsg.com/products/forensic-write-blocking-drive-docks
38. Hardware Write Block | NIST, accessed on September 5, 2025, https://www.nist.gov/itl/ssd/software-quality-group/computer-forensics-tool-testing-program-cftt/cftt-technical/hardware
39. Write Blockers - Digital Intelligence, accessed on September 5, 2025, https://digitalintelligence.com/store/t/solutions/forensic-imaging/write-blockers
40. How to Boot Scan a Mac with APFS and FileVault 2 - ADF Solutions, accessed on September 5, 2025, https://www.adfsolutions.com/news/boot-scan-mac-apfs-filevault-2
41. How To Boot Scan A Mac With APFS And FileVault 2 - Forensic Focus, accessed on September 5, 2025, https://www.forensicfocus.com/articles/how-to-boot-scan-a-mac-with-apfs-and-filevault-2/
42. How to revive or restore Mac firmware - Apple Support, accessed on September 5, 2025, https://support.apple.com/en-us/108900
43. Mac mini m4 stuck in DFU MODE after update and factory reset - Reddit, accessed on September 5, 2025, https://www.reddit.com/r/mac/comments/1lzmxd9/mac_mini_m4_stuck_in_dfu_mode_after_update_and/
44. UEFI Memory Forensics: A Framework for UEFI Threat Analysis - arXiv, accessed

on September 5, 2025, https://arxiv.org/html/2501.16962v1
45. iOS Bootup Process and Jailbreaking | by Syed Asad Ali | Medium, accessed on September 5, 2025, https://medium.com/@syed4asad4/ios-bootup-process-and-jailbreaking-a0b98de5f16c
46. Digital Forensic Analysis on iDevice: Jailbreak iOS 12.1.1 as a Case Study - Telkom University, accessed on September 5, 2025, https://socjs.telkomuniversity.ac.id/ojs/index.php/indojc/article/download/349/152/1590
47. iBoot - Wikipedia, accessed on September 5, 2025, https://en.wikipedia.org/wiki/IBoot
48. iPhone Forensics - On iOS 5 - Exploit DB, accessed on September 5, 2025, https://www.exploit-db.com/docs/english/18400-iphone-forensics-on-ios-5.pdf
49. Here's the Apple Silicon Mac Startup Process based on a DTK firmware analysis (extracted from universal Big Sur Installer) : r/MacOSBeta - Reddit, accessed on September 5, 2025, https://www.reddit.com/r/MacOSBeta/comments/jn7wbf/heres_the_apple_silicon_mac_startup_process_based/
50. Researchers Launch Open-Source UEFI Memory Forensics Framework to Counter Advanced Bootkits - GBHackers, accessed on September 5, 2025, https://gbhackers.com/researchers-launch-open-source-uefi-memory-forensics-framework/
51. Understanding AFU vs BFU in iPhone Forensics and Encryption - SalvationDATA, accessed on September 5, 2025, https://www.salvationdata.com/knowledge/understanding-afu-vs-bfu-in-iphone-forensics/
52. Extract Firmware from OS X installer - GitHub Gist, accessed on September 5, 2025, https://gist.github.com/startergo/18e7fc4f2b1125e80b677181cc6c77e7
53. Apple EFI firmware passwords and the SCBO myth | Reverse Engineering, accessed on September 5, 2025, https://reverse.put.as/2016/06/25/apple-efi-firmware-passwords-and-the-scbo-myth/
54. LongSoft/UEFITool: UEFI firmware image viewer and editor - GitHub, accessed on September 5, 2025, https://github.com/LongSoft/UEFITool
55. binarly-io/fwhunt-scan: Tools for analyzing UEFI firmware and checking UEFI modules with FwHunt rules - GitHub, accessed on September 5, 2025, https://github.com/binarly-io/fwhunt-scan
56. Analyze a firmware image with the firmware analysis service. - Microsoft Learn, accessed on September 5, 2025, https://learn.microsoft.com/en-us/azure/firmware-analysis/tutorial-analyze-firmware
57. Firmware (UEFI) scanning in Defender for Endpoint - Microsoft Learn, accessed on September 5, 2025, https://learn.microsoft.com/en-us/defender-endpoint/uefi-scanning-in-defender-for-endpoint

58. How To Hunt For UEFI Malware Using Velociraptor | Rapid7 Blog, accessed on September 5, 2025, https://www.rapid7.com/blog/post/2024/02/29/how-to-hunt-for-uefi-malware-using-velociraptor/
59. Extracting and Analyzing Apple sysdiagnose Logs - ElcomSoft blog, accessed on September 5, 2025, https://blog.elcomsoft.com/2025/06/extracting-and-analyzing-apple-unified-logs/
60. macOS Forensics 101. It's a Trap! | by Jake Peralta | Aug, 2025 ..., accessed on September 5, 2025, https://medium.com/@jakeperalta7/macos-forensics-101-290fa7c8201a
61. Making Sense of macOS Logs(Part1): A User-Friendly Guide | by Dean - Medium, accessed on September 5, 2025, https://medium.com/@cyberengage.org/making-sense-of-macos-logs-part1-a-user-friendly-guide-8f367b388184
62. Unlocking Unified Logs in Mac Forensics : SUMURI, accessed on September 5, 2025, https://sumuri.com/unlocking-the-power-of-unified-logs-in-mac-forensics/
63. Introduction to macOS Unified Logs - Mac Mini Vault, accessed on September 5, 2025, https://www.macminivault.com/faq/introduction-to-macos-unified-logs/
64. Mac Logging and the log Command: A Guide for Apple Admins - Kandji, accessed on September 5, 2025, https://www.kandji.io/blog/mac-logging-and-the-log-command-a-guide-for-apple-admins
65. Unified Logging for macOS - Jamf Protect Documentation, accessed on September 5, 2025, https://learn.jamf.com/en-US/bundle/jamf-protect-documentation/page/Unified_Logging.html
66. How to Leverage Apple Unified Log (AUL) for IR | CrowdStrike, accessed on September 5, 2025, https://www.crowdstrike.com/en-us/blog/how-to-leverage-apple-unified-log-for-incident-response/
67. Where do I find history log of USB connected devices on my MacBook Pro OS 10.12.6?, accessed on September 5, 2025, https://www.reddit.com/r/computerforensics/comments/7x9279/where_do_i_find_history_log_of_usb_connected/
68. Is there a system log which displays whic... - Apple Support Communities, accessed on September 5, 2025, https://discussions.apple.com/thread/6734878
69. FSEvents: How They Work and Why They Matter for Mac Analysis ..., accessed on September 5, 2025, https://www.hexordia.com/blog/mac-forensics-analysis
70. Leveraging FSEvents for Mac Forensic Analysis - HackMD, accessed on September 5, 2025, https://hackmd.io/@M4shl3/FSEvents
71. macOS Forensic Artifacts - Cyber Social Hub, accessed on September 5, 2025, https://cybersocialhub.com/post/macos-forensic-artifacts
72. How to Fix MDS Stores High CPU on Mac | Trend Micro News, accessed on

September 5, 2025, https://news.trendmicro.com/2023/06/13/mds-stores-high-cpu-mac-how-to-fix/
73. Apple Computer Forensics, accessed on September 5, 2025, https://www.dex-forensics.com/apple-computer-forensics-1
74. Spotlight database file format - - Forensics Wiki, accessed on September 5, 2025, https://forensics.wiki/spotlight_database_file_format/
75. Rebuild the Spotlight index on your Mac - Apple Support, accessed on September 5, 2025, https://support.apple.com/en-us/102321
76. 7 essential artifacts for macOS forensics, accessed on September 5, 2025, https://www.magnetforensics.com/blog/essential-artifacts-for-macos-forensics/
77. From Backup to Breakthrough: APFS Snapshots in Digital Forensics - NBFTools, accessed on September 5, 2025, https://www.nbftools.com/apfs-snapshots-in-digital-forensics/
78. Manage Time Machine backups | Macintosh notes - GitHub, accessed on September 5, 2025, https://github.molgen.mpg.de/pages/bs/macOSnotes/mac/mac_files_tmutil.html
79. Timemachine 4n6 - Musings on 4n6 - WordPress.com, accessed on September 5, 2025, https://d4rkw1ll0w4n6.wordpress.com/2015/02/12/timemachine-4n6/
80. The Art of Mac Malware, Volume 2, accessed on September 5, 2025, https://taomm.org/vol2/pdfs/CH%205%20Persistence.pdf
81. Mac os x 10.9 artifacts location - - Forensics Wiki, accessed on September 5, 2025, https://forensics.wiki/mac_os_x_10.9_artifacts_location/
82. Tools - Objective-See, accessed on September 5, 2025, https://objective-see.org/tools.html
83. MacOS.System.QuarantineEvents :: Velociraptor - Digging deeper!, accessed on September 5, 2025, https://docs.velociraptor.app/artifact_references/pages/macos.system.quarantineevents/
84. File Sentinel: Shell History 4+ - App Store, accessed on September 5, 2025, https://apps.apple.com/us/app/file-sentinel-shell-history/id6744690194?mt=12
85. See full history in Mac Terminal. with .zshrc | by Niki Trivedi | Medium, accessed on September 5, 2025, https://medium.com/@trivediniki94/see-full-history-in-mac-terminal-8e494bada345
86. KnowledgeC Database Forensics: A Comprehensive Guide - Belkasoft, accessed on September 5, 2025, https://belkasoft.com/knowledgec-database-forensics-with-belkasoft
87. How to Analyze KnowledgeC.db with Belkasoft X, accessed on September 5, 2025, https://belkasoft.com/how-to-analyze-knowledgec-database-with-belkasoft-x
88. The KnowledgeC.db File within Apple Devices - Athena Forensics, accessed on September 5, 2025, https://athenaforensics.co.uk/the-knowledgec-db-within-apple-devices/
89. Keychain (software) - Wikipedia, accessed on September 5, 2025, https://en.wikipedia.org/wiki/Keychain_(software)

90. Keychain data protection - Apple Support, accessed on September 5, 2025, https://support.apple.com/guide/security/keychain-data-protection-secb0694df1a/web
91. MAC OS X Forensics: Password Discovery - Scholarly Commons, accessed on September 5, 2025, https://commons.erau.edu/adfsl/2011/wednesday/5/
92. macOS Forensics 102. The Joy of Hidden Plists | by Jake Peralta | Aug, 2025 | Medium, accessed on September 5, 2025, https://medium.com/@jakeperalta7/macos-forensics-102-5b65acb0468d
93. Mac USB History - Forensic Focus, accessed on September 5, 2025, https://www.forensicfocus.com/forums/general/mac-usb-history/
94. FOR518: Mac and iOS Forensic Analysis and Incident Response - SANS Institute, accessed on September 5, 2025, https://www.sans.org/cyber-security-courses/mac-and-ios-forensic-analysis-and-incident-response
95. mac4n6.com, accessed on September 5, 2025, http://www.mac4n6.com/
96. USB Forensic Analysis: Reveal Every USB Device That Has Connected to Your Computer, accessed on September 5, 2025, https://www.examcollection.com/blog/usb-forensic-analysis-reveal-every-usb-device-that-has-connected-to-your-computer/
97. USB Forensics Unveiled: Discover the Hidden Record of Every Device Ever Plugged In, accessed on September 5, 2025, https://www.examcollection.com/blog/usb-forensics-unveiled-discover-the-hidden-record-of-every-device-ever-plugged-in/
98. How to query the history record of when the USB driver which has been used?, accessed on September 5, 2025, https://apple.stackexchange.com/questions/242362/how-to-query-the-history-record-of-when-the-usb-driver-which-has-been-used
99. Can I view history of recently connected USB devices in OS X? - Apple Stack Exchange, accessed on September 5, 2025, https://apple.stackexchange.com/questions/226816/can-i-view-history-of-recently-connected-usb-devices-in-os-x
100. How to See Thunderbolt and USB4 Device Information on a Mac - ServeTheHome, accessed on September 5, 2025, https://www.servethehome.com/how-to-see-thunderbolt-and-usb4-device-information-on-a-mac/
101. MacBook Pro OS X Connected Device History : r/computerforensics - Reddit, accessed on September 5, 2025, https://www.reddit.com/r/computerforensics/comments/4w6jou/macbook_pro_os_x_connected_device_history/
102. mesquidar/ForensicsTools: A list of free and open forensics analysis tools and other resources - GitHub, accessed on September 5, 2025, https://github.com/mesquidar/ForensicsTools
103. A curated list of awesome forensic analysis tools and resources - GitHub, accessed on September 5, 2025, https://github.com/cugu/awesome-forensics
104. Digital Forensic Timeline Analysis with Belkasoft X, accessed on September 5,

2025, https://belkasoft.com/digital-forensic-timeline-analysis
105. Timeline Explorer - SANS Institute, accessed on September 5, 2025, https://www.sans.org/tools/timeline-explorer
106. Malware Analysis: Steps & Examples - CrowdStrike, accessed on September 5, 2025, https://www.crowdstrike.com/en-us/cybersecurity-101/malware/malware-analysis/
107. #OBTS v6.0: The Art of Mac Malware - Objective by the Sea, accessed on September 5, 2025, https://objectivebythesea.org/v6/taomm.html
108. rshipp/awesome-malware-analysis: Defund the Police. - GitHub, accessed on September 5, 2025, https://github.com/rshipp/awesome-malware-analysis
109. Mac Malware | Red Canary Threat Detection Report, accessed on September 5, 2025, https://redcanary.com/threat-detection-report/trends/mac-malware/
110. chain of custody - Glossary | CSRC, accessed on September 5, 2025, https://csrc.nist.gov/glossary/term/chain_of_custody
111. Forms & Templates - DFIR Training, accessed on September 5, 2025, https://www.dfir.training/downloads/forms-templates?category[0]=2&category_children=1&limit=20&limitstart=20